

Software Architecture and Design Specification

Vehicle Parking System

Group 14 — Problem Statement No. 10

Project	Vehicle Parking System (MERN Stack)
Document	Software Architecture and Design Specification (SAD)
Date	03-10-2026
References	Vehicle Parking System — Software Requirements Specification v1.0 Vehicle Parking System — Software Test Plan v1.0
Authors	Siddhesh Dattatray Koditkar — PES2UG24CS502 — Project Lead & Development Lead Sumukh M Gowda — PES2UG24CS537 — Backend & Integration Lead Sparsha Arun — PES2UG24CS513 — Frontend & Test Automation Lead Sinchana Kulkarni — PES2UG24CS504 — Security & Release Lead

Table of Contents

- 1. Introduction
- 2. Document Overview
- 3. Architecture
 - 3.1 Goals & Constraints 3.2 Stakeholders & Concerns 3.3 Component Diagram
 - 3.4 Component Descriptions 3.5 Architecture Pattern & Rationale 3.6 Technology Stack
 - 3.7 Risks & Mitigations 3.8 Traceability to Requirements 3.9 Security Architecture
- 4. Design
 - 4.1 Design Overview 4.2 UML Sequence Diagrams 4.3 API Design
 - 4.4 Error Handling, Logging & Monitoring 4.5 UX Design 4.6 Open Issues & Next Steps
- 5. Appendices (Glossary, References, Tools)

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the technical architecture and design of the Vehicle Parking System (VPS) v1.0. It translates the requirements in the VPS SRS v1.0 into a concrete structure — components, data flow, design patterns, API contracts, and security architecture — implementable by the team and by AI coding agents assisting with development.

1.2 Scope

This document covers backend and frontend component architecture, inter-component data flow, the chosen architecture pattern and its rationale, API design, security/threat modeling, and UX design principles for VPS v1.0. It does not contain implementation-level code; verification strategy is defined separately in the VPS STP v1.0.

1.3 Audience

Developers, QA and security reviewers.

1.4 Definitions, Acronyms and Abbreviations

- VPS — Vehicle Parking System
- SAD — Software Architecture and Design Specification
- ADR — Architecture Decision Record
- RBAC — Role-Based Access Control
- JWT — JSON Web Token
- FCFS — First-Come, First-Served
- ODM — Object Document Mapper (Mongoose)
- REST — Representational State Transfer
- STRIDE — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (threat-modeling framework)

2. Document Overview

2.1 How to Use This Document

Section 3 (Architecture) establishes the system's structure: components, chosen pattern, technology stack, and security model, each traced back to specific SRS requirement IDs. Section 4 (Design) provides the sequence-level behavior, full API contracts, and UX principles. Major architectural decisions (e.g. the multi-tab simulation approach, the atomic-allocation strategy) are recorded as short rationale notes within Section 3.5 rather than as separately numbered ADRs, since this is the natural location for them in the template.

2.2 Related Documents

- Vehicle Parking System — Software Requirements Specification (SRS), v1.0 — defines what the system must do
- Vehicle Parking System — Software Test Plan (STP), v1.0 — defines how it is verified

3. Architecture

3.1 Goals & Constraints

Goals:

- Concurrency-safety — zero duplicate spot allocations under simultaneous multi-gate load.
- Real-time cross-session consistency — every open session reflects the same state within 2 seconds.
- Strict role separation — Gate Operators and Admin see and can do materially different things, enforced server-side.
- Testability — architecture supports the STP's four-level test strategy and limits the blast radius of subtle errors in AI-agent-generated code.

Constraints:

- MERN stack only (MongoDB, Express, React, Node.js).
- No physical hardware or payment-gateway integration (see SRS Section 2.5).
- Must run as a local, single-host simulation — multiple browser tabs standing in for physically separate terminals.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concern
Gate Operators	Fast, simple, per-gate-scoped interface — minimal clicks per transaction
Admin / Manager	Full visibility (analytics, raw records, audit log) and override control
Course Instructor	Demonstrable software-engineering rigor and testability
Development Team	A maintainable architecture that stays safe to extend with AI coding assistance

3.3 Component (UML) Diagram

Source: diagrams/component_diagram.puml

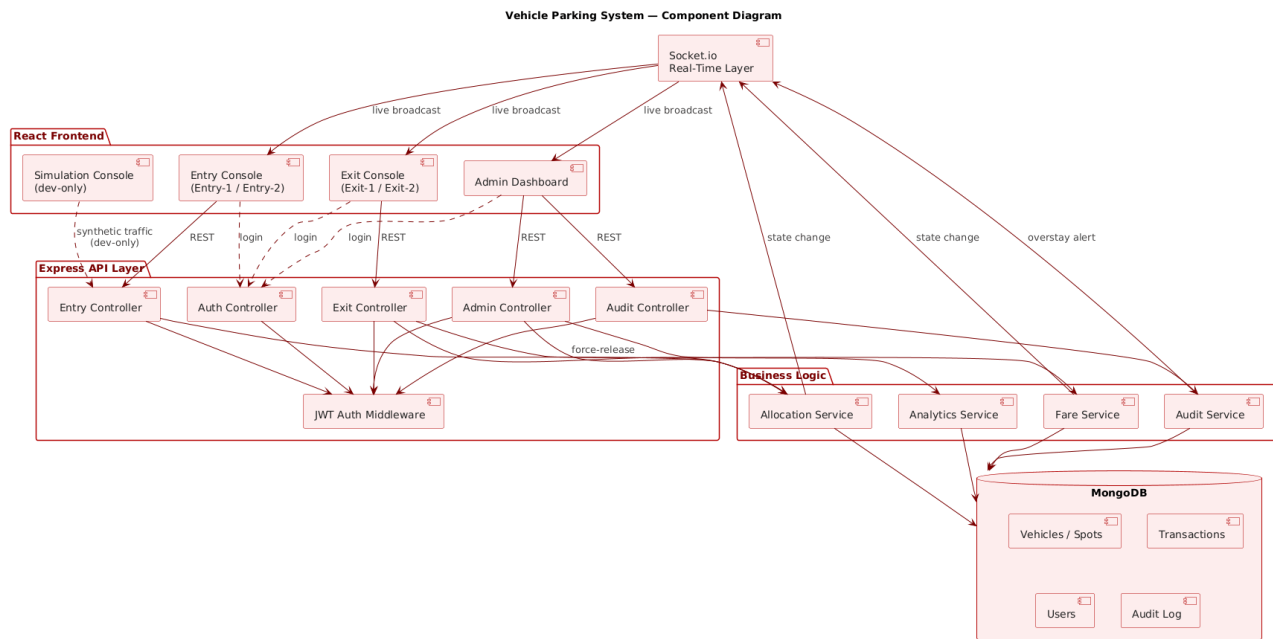


Figure 1 — Component Diagram

3.4 Component Descriptions

Component	Responsibility
React Frontend (Entry / Exit / Admin / Simulation consoles)	Presentation layer — one SPA serving role-specific views based on the authenticated user's JWT claims
Express API Layer (Auth / Entry / Exit / Admin / Audit Controllers + JWT Middleware)	REST request handling and authorization enforcement; rejects any request without a valid, correctly-scoped token
Business Logic (Allocation / Fare / Analytics / Audit Services)	Core domain logic, deliberately independent of HTTP or Socket.io so it can be unit-tested in isolation (STP 5.1)
Socket.io Real-Time Layer	Broadcasts state changes to every connected client — the mechanism behind the multi-tab live-sync demo
MongoDB (Vehicles/Spots, Transactions, Users, Audit Log)	Persistent store, accessed only through the Business Logic layer, never directly from a controller

3.5 Chosen Architecture Pattern and Rationale

Pattern: a layered architecture (presentation → API → business logic → data) with a publish/subscribe overlay (Socket.io) for real-time concerns. The layering keeps business logic testable in isolation — a unit test can exercise the Fare Service's tier calculation without an HTTP server or a socket connection. A monolithic single Express process was chosen over microservices: microservices were rejected as unnecessary operational complexity.

Key decisions (recorded here as short rationale notes, standing in for formal ADRs — the template has no separate ADR section):

- Multi-tab-as-terminals simulation — chosen over a genuinely distributed deployment because it fully demonstrates concurrency and real-time sync without requiring real networking or hardware (SRS Section 2.1).
- Atomic MongoDB findOneAndUpdate for allocation — chosen over an in-memory lock or a message queue because it is the simplest mechanism that still gives a hard correctness guarantee, and it is exactly what VPS-F-012 and VPS-NF-002 require.
- Simulation Console kept outside the role model — chosen to keep the deployed system's role model honest (SRS Section 2.6) rather than inventing a fictitious stakeholder for a developer convenience tool.

3.6 Technology Stack & Data Stores

- Frontend — React (SPA), socket.io-client
- Backend — Node.js 24 LTS, Express, Socket.io
- Database — MongoDB, accessed via Mongoose ODM
- Auth — JWT for session tokens, bcrypt for password hashing
- Testing — Jest, Supertest, React Testing Library, k6 (per STP Section 6)

3.7 Risks & Mitigations

Risk	Mitigation
Single Node.js process is a single point of failure.	MongoDB persistence means a restart doesn't lose state (VPS-NF-003).
Concurrency bugs in allocation logic.	Atomic database operations plus a dedicated k6 load test (STP Section 7, Week 4).
Socket.io state drifts between tabs after a reconnect.	Server sends a full lot:snapshot on every new connection, not just incremental deltas.

3.8 Traceability to Requirements

Requirement IDs	Component(s)
VPS-F-001..005, PRJ-SR-002/003	Auth Controller + JWT Middleware
VPS-F-010..016	Entry Controller + Allocation Service
VPS-F-020..023, VPS-NF-001	Socket.io Real-Time Layer
VPS-F-030..037	Exit Controller + Fare Service
VPS-F-040..043	Admin Controller + Allocation Service + Fare Service
VPS-F-050..056	Admin Controller + Analytics Service
VPS-F-060..062, PRJ-SR-006	Audit Controller + Audit Service
VPS-NF-002	Allocation Service (atomic MongoDB operations)
PRJ-SR-001..006	JWT Middleware, Auth Controller, Audit Service (cross-cutting)

3.9 Security Architecture

Threat model (STRIDE), mapped to the SRS's security requirements:

Threat	Mitigation
Spoofing	Gate-scoped JWT issuance and validation on every protected route (PRJ-SR-002, PRJ-SR-004)
Tampering	All writes go through validated, atomic server-side operations only — no client ever writes directly to MongoDB
Repudiation	Append-only audit log for every state-changing action (PRJ-SR-006)
Information Disclosure	RBAC — Gate Operators cannot reach admin-only endpoints or raw transaction records (PRJ-SR-005)
Denial of Service	Account lockout after 5 consecutive failed login attempts (VPS-F-003); broader DoS protection is out of scope for a course-scale local simulation
Elevation of Privilege	Role check enforced server-side on every protected route, never relied on as UI-only (PRJ-SR-005)

4. Design

4.1 Design Overview

VPS is designed as a layered backend (controllers → business-logic services → MongoDB) with a Socket.io publish/subscribe layer running alongside it for real-time concerns (Section 3.5). The sequence diagrams below show how that structure behaves for the system's three defining flows: concurrent multi-gate entry allocation, exit with fare calculation, and overstay escalation to the Admin.

4.2 UML Sequence Diagrams

Three sequence diagrams are provided, covering the flows most central to this project's architecture — concurrency-safe allocation being the flagship one. Each is available as rendered output here and as PlantUML (.puml) source in the accompanying diagrams/ folder.

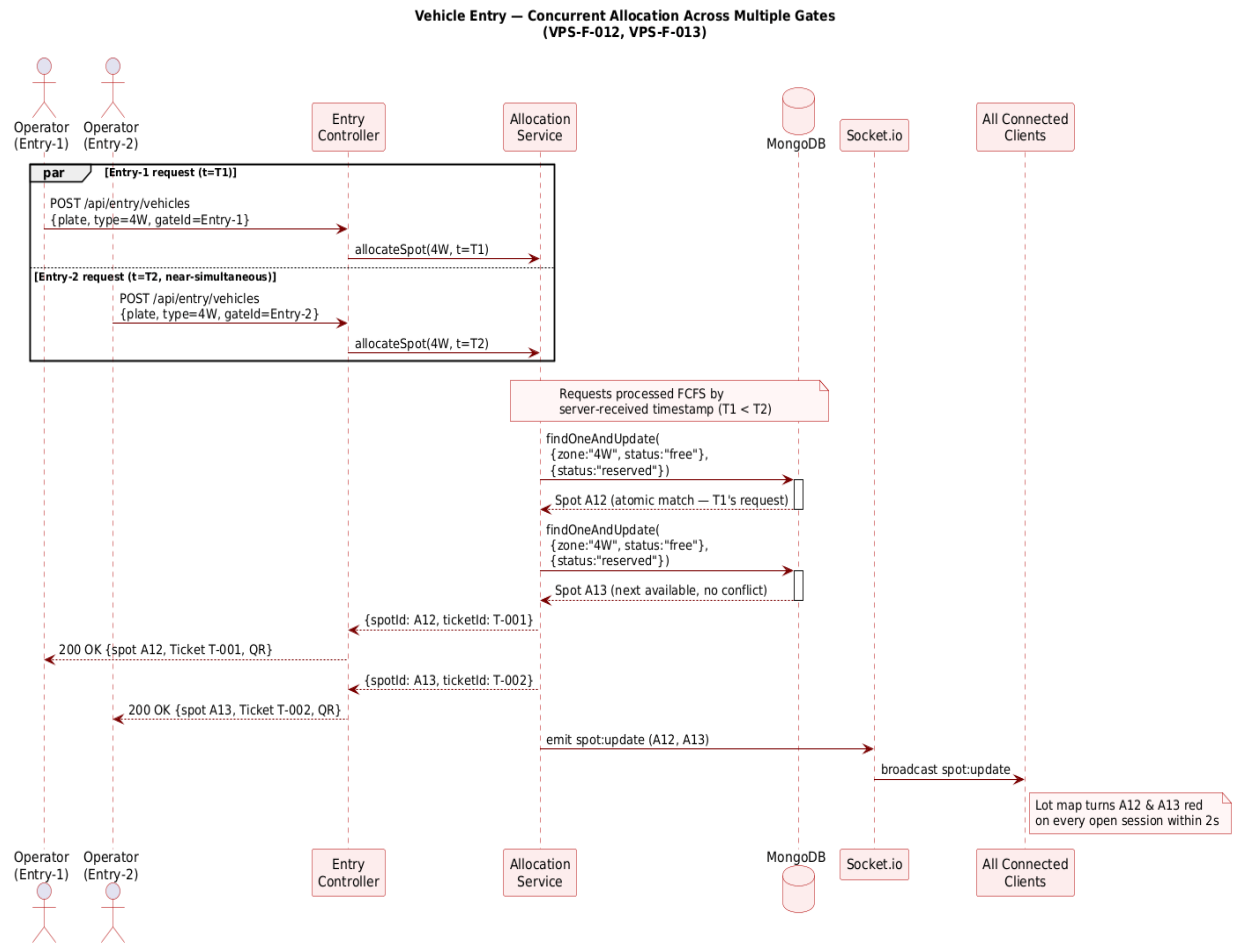


Figure 2 — Vehicle Entry: Concurrent Allocation Across Multiple Gates (VPS-F-012, VPS-F-013)

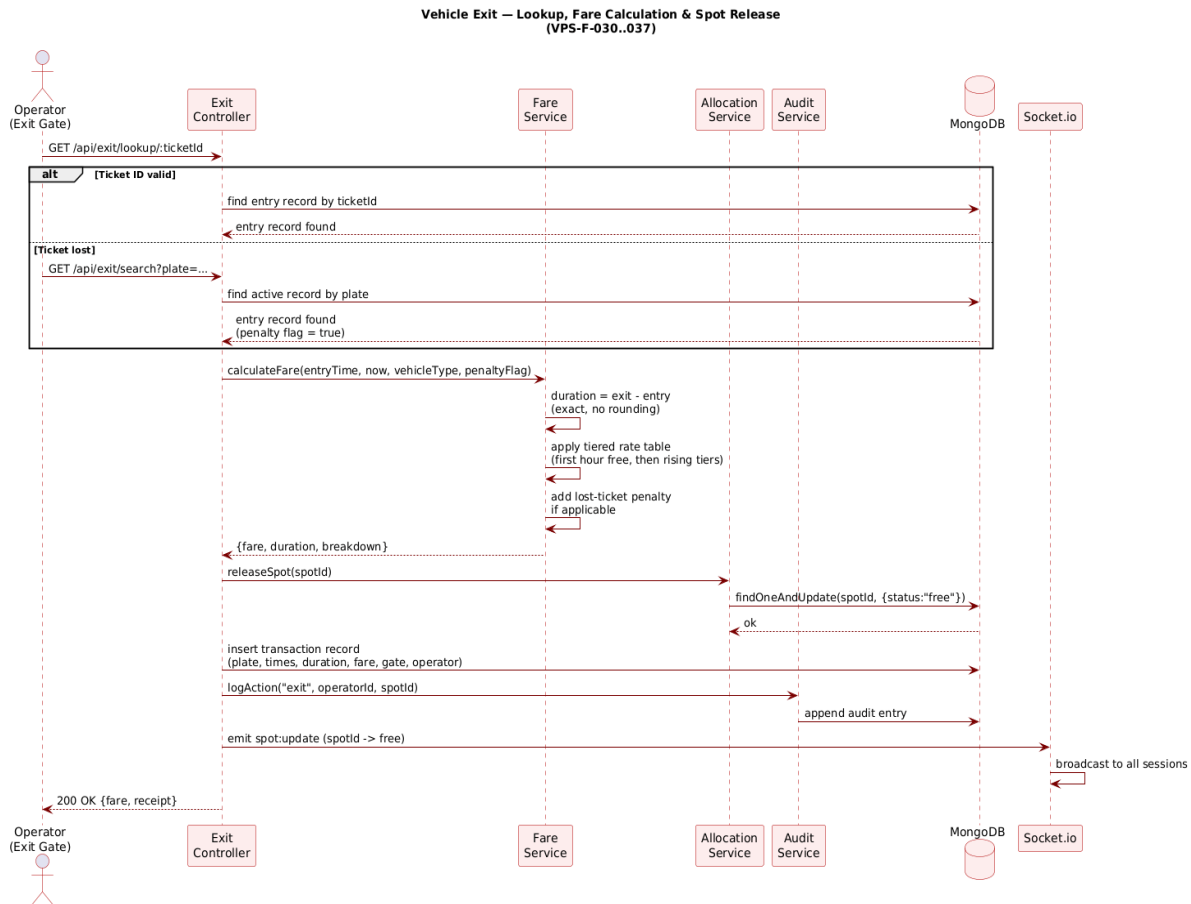


Figure 3 — Vehicle Exit: Lookup, Fare Calculation & Spot Release (VPS-F-030..037)

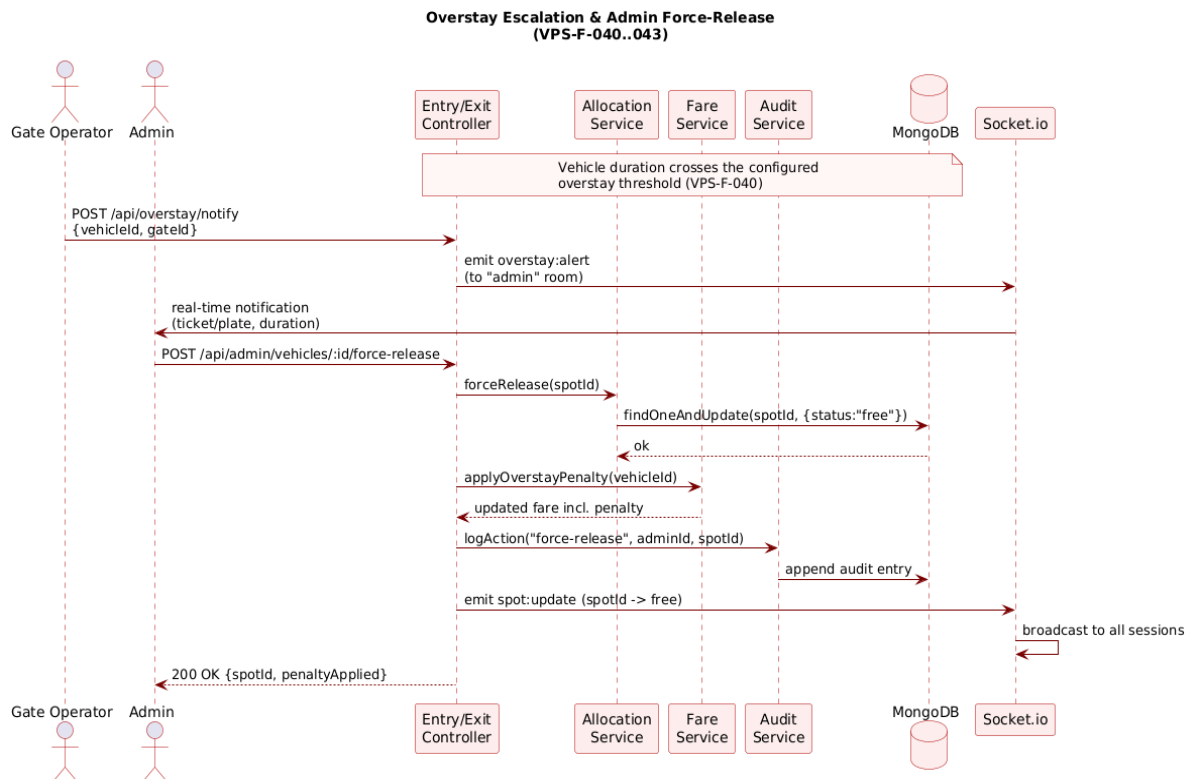


Figure 4 — Overstay Escalation & Admin Force-Release (VPS-F-040..043)

4.3 API Design

Comprehensive endpoint list, grouped by module and mapped to SRS requirement IDs, so the contract and the requirement it satisfies are never more than one table away from each other.

4.3.1 Authentication

Endpoint	Request	Response	Errors	Implements
POST /api/auth/login	{username, password}	{token, role, gateId?}	401 Invalid credentials 423 Account locked	VPS-F-001..005 PRJ-SR-002/003

4.3.2 Vehicle Entry & Allocation

Endpoint	Request	Response	Errors	Implements
GET /api/lot/status	query: zone (optional)	{spots:[{id,zone,status}], occupancy}	401 Unauthorized	VPS-F-020, VPS-F-022
POST /api/entry/vehicles	{plateNumber, vehicleType, gateId}	{ticketId, spotId, zone, qrPayload}	401 Unauthorized gate 409 Lot Full	VPS-F-010..016 PRJ-SR-004

4.3.3 Vehicle Exit & Fare

Endpoint	Request	Response	Errors	Implements
GET /api/exit/lookup/:ticketId	path: ticketId	{entry record}	404 Not found	VPS-F-030
GET /api/exit/search	query: plateNumber	{matching active record(s)}	404 Not found	VPS-F-031
POST /api/exit/process	{ticketId? , plateNumber?, gateId}	{fare, durationSeconds, penaltyApplied, breakdown, receiptId}	401 Unauthorized gate 404 Not found 409 Already exited	VPS-F-032..037

4.3.4 Overstay & Admin Override

Endpoint	Request	Response	Errors	Implements
POST /api/overstay/notify	{vehicleId, gateId}	{notificationId}	404 Vehicle not found	VPS-F-040, VPS-F-041
POST /api/admin/vehicles/:id/force-release	path: id	{spotId, penaltyApplied}	403 Forbidden 404 Not found	VPS-F-042, VPS-F-043 PRJ-SR-005

4.3.5 Admin Analytics & Configuration

Endpoint	Request	Response	Errors	Implements
GET /api/admin/analytics/occupancy	—	{totalPct, byZone, history[]}	403 Forbidden	VPS-F-051, VPS-F-053
GET /api/admin/analytics/revenue	—	{total, byDay[], byZone}	403 Forbidden	VPS-F-052
GET /api/admin/transactions	query: plate, dateRange, gate	{records[]}	403 Forbidden	VPS-F-054
PUT /api/admin/config/fare-tiers	{vehicleType, tiers: [{afterHours,rate}]}	{updated:true}	403 Forbidden 400 Invalid tiers	VPS-F-055

Endpoint	Request	Response	Errors	Implements
PUT /api/admin/config/spots	{zone, count}	{updated:true}	403 Forbidden	VPS-F-056

4.3.6 Audit

Endpoint	Request	Response	Errors	Implements
GET /api/admin/audit-log	query: action, actor, dateRange	{entries[]}	403 Forbidden	VPS-F-060, VPS-F-061

4.3.7 Simulation Console (dev-only — excluded from deployed scope)

Endpoint	Request	Response	Errors	Implements
POST /api/dev/simulate/start	{ratePerMinute}	{status:"running"}	—	VPS-DEV-001, VPS-DEV-002
POST /api/dev/simulate/stop	—	{status:"stopped"}	—	VPS-DEV-003

4.3.8 Real-Time Events (Socket.io — not REST)

Event	Direction	Payload	Implements
identify	Client → Server	{token} — authenticates the socket and assigns it to a role-based room	PRJ-SR-002, PRJ-SR-005
lot:snapshot	Server → Client	Full current spot map, sent on connect	VPS-F-022
spot:update	Server → Client	{spotId, status}	VPS-F-021, VPS-F-022, VPS-F-036
overstay:alert	Server → Client ("admin" room only)	{vehicleId, gateId, duration}	VPS-F-041

4.4 Error Handling, Logging & Monitoring

- Standard error envelope on every failed response: {error: {code, message}} — HTTP status mirrors the code.
- No sensitive data (passwords, raw JWTs) ever appears in logs (mirrors STP 5.1 / PRJ-SR-003).
- Monitoring metrics: allocation-request latency, Socket.io broadcast latency, duplicate-allocation count (the primary concurrency health signal — should always read zero), and failed-login count (security signal).

4.5 UX Design

- Minimalist red-and-white theme, no decorative animation (SRS 3.1 / 4.3).
- Spot status conveyed by colour and a text/icon label — not colour alone — so the lot map isn't purely colour-dependent.
- Operator flows kept to at most 3 interactions after login (VPS-NF-004).
- Admin dashboard groups the live map, analytics, and audit log into clearly separated panels, so the Admin's broader visibility doesn't visually overwhelm the simpler operator views.

4.6 Open Issues & Next Steps

- Real ANPR/camera-based plate capture (currently manual entry) — potential future enhancement, out of current scope.
- Real payment-gateway integration (fare is currently calculated and displayed only) — potential future enhancement.
- Customer-facing booking portal — explicitly rejected for this project (SRS Section 1.2); could be a future extension if requirements change.

5. Appendices

5.1 Glossary

VPS, SAD, SRS, STP, ADR, RBAC, JWT, FCFS, ODM, REST, STRIDE — see Section 1.4 for definitions.

5.2 References

- Vehicle Parking System — Software Requirements Specification (SRS), v1.0
- Vehicle Parking System — Software Test Plan (STP), v1.0
- OWASP — general guidance referenced for the security architecture in Section 3.9

5.3 Tools

- Jest, Supertest, React Testing Library — automated testing (per STP Section 6)
- k6 — concurrency/load testing (per STP Section 6)
- Postman — manual API testing
- Git / GitHub — version control